P4

Contrastar hipótesis con GDB

Parte I · RA4 · 4 h

Universidad de Almería · Edición de 27 de septiembre de 2026

Documento: Guion de práctica para el alumnado
Asignatura: Ingeniería Inversa y Análisis Forense Digital
Uso: Laboratorio docente en entorno controlado y sobre las muestras indicadas
Alumno/a: _____ Grupo: _____

Índice

1. Mapa de trabajo **2**

2. Material **2**

3. Trabajo en sesión (240 minutos) **2**

4. Inicio reproducible (desde la raíz) **2**

5. Entregable **2**

6. Evaluación **3**

7. Teoría necesaria para realizar la práctica **3**

8. Software requerido **3**

9. Guía detallada de realización **3**
9.1. 1. Crear el control sin depurador . . . 3
9.2. 2. Configurar logging de GDB . . . 3
9.3. 3. Observar argumentos, memoria y retorno . . . 4
9.4. 4. Resolver PIE/ASLR en RE-01 . . . 4
9.5. 5. Ejecutar las cinco pruebas . . . 4
9.6. 6. Intervenir y controlar . . . 4
9.7. 7. Actualizar la especificación . . . 4

10. Seguridad y solución de problemas **5**

11. Entrega desarrollada **5**

12. Actualización RE-01 1.0 — 28 de septiembre de 2026 **5**
12.1. Control y cinco entradas concretas . . . 5
12.2. Observar el ELF stripped y resolver su base . . . 5
12.3. Trazado de efectos e intervención . . . 6

Estado actual: RE-01 1.0 disponible. La actualización del 28 de septiembre de 2026, al final del documento, contiene el material definitivo y sustituye las menciones a RE-01 pendiente de producción. Las fichas anteriores se conservan como antecedente.

1 Mapa de trabajo



Figura 1: Secuencia de trabajo y puntos de control de la práctica.

El diagrama resume el orden recomendado. Cada transición debe quedar respaldada por una anotación, salida o fichero verificable en la entrega.

4 h · RA4 · 15 % de la nota de Parte I. Guion completo en LaTeX. Rúbrica única en criterios de evaluación.

2 Material

Muestra de inicio: `ejemplos-x86-dbg`; núcleo: RE-01 y predicciones de P3 (pendiente el paquete del caso). VM aislada, copia de trabajo y carpeta de resultados nueva.

3 Trabajo en sesión (240 minutos)

1. Control sin intervención y preparación de registros (20 min).
2. GDB: observar argumentos, retorno, memoria y flujo en muestra con símbolos (45 min).
3. Resolver una dirección de RE-01 con PIE/load bias; documentar ASLR y mapeos (35 min).
4. Ejecutar las cinco entradas previstas en P3 y contrastar resultados, errores y efectos con GDB/strace (75 min).
5. En el crackme, alterar una decisión después de observar el retorno y compararla con el control; repetir sin intervención (35 min).
6. Actualizar la especificación y explicar una hipótesis refutada (30 min).

4 Inicio reproducible (desde la raíz)

```
gdb --args muestras/build/salida/ejemplos-x86-dbg AAAAAAAAAAAAA
```

Dentro de GDB: `break validar_contrasena`, `run`, `disassemble /r`, `info args`, `x/s intento`, `finish`, `info registers eax`. Guardar la sesión con logging. Los breakpoints en offsets se eligen a partir del desensamblado actual. No asumir que un `+N` de otra versión apunta a la misma instrucción ni desactivar ASLR globalmente.

5 Entregable

Matriz predicción→observación→explicación para cinco entradas, sesión reproducible, registro de efectos y distinción entre ejecución normal e intervenida. Forzar éxito no significa encontrar una entrada válida ni demostrar actividad histórica.

6 Evaluación

Se aplica la rúbrica común: corrección técnica 40 %, evidencia reproducible 25 %, interpretación y límites 25 %, claridad 10 %. Todos los criterios se aplican al objetivo concreto de esta práctica; las ampliaciones no son necesarias para la máxima nota.

7 Teoría necesaria para realizar la práctica

El análisis dinámico observa una ejecución concreta. Una prueba confirma o refuta una predicción bajo una entrada, versión y entorno determinados; no prueba todas las ejecuciones posibles ni actividad histórica. Un experimento útil define antes: hipótesis, entrada, estado inicial, punto de observación, esperado y control.

GDB modifica tiempos, señales y estado del proceso. Un breakpoint de software suele insertar temporalmente `int3`; los registros cambiados por el analista producen una ejecución intervenida, que se documenta separada del control sin cambios.

Con PIE y ASLR, las direcciones del fichero son relativas a una base de carga. Se leen `info files`, `info proc mappings` y el rango del build actual. No se desactiva ASLR globalmente ni se reutilizan offsets de otra compilación.

8 Software requerido

GDB, `strace`, `file`, `sha256sum`, GNU Make y resultados de P3.

```
bash PreparaciónEntorno/entorno/provision.sh --check
gdb --version | head -n 1
strace -V 2>&1 | head -n 1
make -C muestras/build x86
mkdir -p resultados/P4
```

La VM debe permitir `ptrace` entre procesos del mismo usuario. No se ejecutan muestras en el host ni sobre datos personales o carpetas compartidas.

9 Guía detallada de realización

9.1 1. Crear el control sin depurador

```
set +e
muestras/build/salida/ejemplos-x86-dbg AAAAAAAAAAAA \
  > resultados/P4/control.out 2> resultados/P4/control.err
printf 'exit=%s\n' "$?" > resultados/P4/control.exit
set -e
sha256sum muestras/build/salida/ejemplos-x86-dbg \
  > resultados/P4/muestra.sha256
```

Registre entrada, stdout, stderr, retorno y efectos. `set +e` permite conservar un retorno 1 esperado; no transforma el resultado del programa.

9.2 2. Configurar logging de GDB

```
gdb -q --args muestras/build/salida/ejemplos-x86-dbg AAAAAAAAAAAA
```

Dentro de GDB:

```
set pagination off
set logging file resultados/P4/gdb-calentamiento.txt
set logging overwrite on
set logging enabled on
break validar_contrasena
run
info args
disassemble /r validar_contrasena
x/s intento
finish
info registers eax
set logging enabled off
quit
```

Si el build no conserva el nombre, localice la función con P3 y use una dirección del binario actual.

9.3 3. Observar argumentos, memoria y retorno

Antes de cada comando escriba qué espera. Compare el puntero de `info args` con `x/s`; avance mediante `nexti` o `stepi`. `finish` permite observar el retorno. Anote qué instrucciones han ejecutado antes de interpretar un registro.

9.4 4. Resolver PIE/ASLR en RE-01

Registre `info files`, `info proc mappings` y la dirección estática elegida en P3. Calcule la efectiva y compruebe instrucciones con `x/8i`. Solo después instale el breakpoint. La coincidencia de instrucciones valida el mapeo.

9.5 5. Ejecutar las cinco pruebas

Para cada entrada de P3 ejecute primero control y después sesión observada:

ID	Predicción	Entrada/estado	Punto observado	Resultado	Efectos	Decisión
T1–T5	completar	completar	dirección/símbolo	completar	completar	confirma/refuta

`strace -f -o resultados/P4/TN.strace - PROGRAMA ARGUMENTOS` ayuda a observar aperturas, lecturas, escrituras y retorno. Una llamada exitosa no demuestra que el contenido sea el esperado: relacione descriptor, longitud y datos cuando proceda.

9.6 6. Intervenir y controlar

En el calentamiento, observe el retorno original. Antes de que el llamante lo consuma:

```
info registers eax
set $eax = 1
continue
```

Repita en un proceso nuevo sin cambio. Se forzó la decisión de esa ejecución; no se halló una entrada válida ni se modificó el fichero.

9.7 7. Actualizar la especificación

Corrija condiciones o tipos de P3 y conserve la versión previa. Una hipótesis refutada mediante un experimento bien diseñado es un resultado válido.

10 Seguridad y solución de problemas

- Si GDB no puede trazar, registre kernel, política y error; no use `sudo` como arreglo.
- Si el breakpoint no se activa, confirme hash, símbolo, base y camino.
- Si hay escrituras, restaure una copia del estado inicial.
- Si `strace` altera el comportamiento, conserve control y documente el efecto.
- No cambie registros antes de guardar su valor original.

11 Entrega desarrollada

Incluya entorno, hash, control, log GDB, cálculo PIE, matriz de cinco pruebas, intervención/control, efectos, especificación corregida y límites. El anexo conserva comandos; el cuerpo selecciona las observaciones que responden a cada hipótesis.

12 Actualización RE-01 1.0 — 28 de septiembre de 2026

El paquete central ya está disponible; esta sección sustituye el estado pendiente de la ficha histórica, conservada arriba. Use el mismo RE-01 y MANIFIESTO.json que en P3. Las pruebas del crackme siguen siendo un calentamiento independiente.

12.1 Control y cinco entradas concretas

Desde alumnado, en una terminal Bash de la VM con GDB y strace:

```
sha256sum -c SHA256SUMS
mkdir sesion-re01
sha256sum RE-01 entradas/* > sesion-re01/antes.sha256
set +e
for caso in T1 T2 T3 T4 T5; do
  case "$caso" in
    T1) cfg=valida.cfg; doc=documento.txt ;;
    T2) cfg=valida.cfg; doc=vacio.txt ;;
    T3) cfg=limite.cfg; doc=limite.txt ;;
    T4) cfg=limite.cfg; doc=exceso.txt ;;
    T5) cfg=opcion-invalida.cfg; doc=documento.txt ;;
  esac
  ./RE-01 "entradas/$cfg" "entradas/$doc" "sesion-re01/$caso.txt" \
    > "sesion-re01/$caso.stdout" 2> "sesion-re01/$caso.stderr"
  printf '%s\n' "$?" > "sesion-re01/$caso.exit"
done
sha256sum -c sesion-re01/antes.sha256
```

No reutilice sesion-re01 en otra ejecución. Cada salida debe ser nueva. Compare los controles con las predicciones RE-01.T1–T5 de P3 y registre existencia y contenido del resultado; stdout y stderr creados por el shell no son ese resultado.

12.2 Observar el ELF stripped y resolver su base

```
gdb -nx -q --args ./RE-01 entradas/valida.cfg \
  entradas/documento.txt sesion-re01/T1-gdb.txt
```

```
set pagination off
set logging file sesion-re01/gdb.txt
set logging enabled on
set disable-randomization off
starti
info files
info proc mappings
```

Relacione un mapeo del propio RE-01 con un PT_LOAD y su offset de fichero usando readelf -lW. Reste la VA del segmento, alineada como el mapeo, a su dirección inicial para obtener load bias. A cada VA ELF reconstruida en P3 súmele esa base. Compruebe instrucciones con x/8i antes de usar break *DIRECCION. starti puede parar en el cargador, cuya base no es la de RE-01. Observe los argumentos según el ABI System V AMD64: rdi/rsi/rdx al entrar en las tres primeras posiciones; eax para retornos uint32_t/int. Registre el punto exacto antes de interpretar cada valor.

12.3 Trazado de efectos e intervención

```
strace -f -o sesion-re01/T1.strace -- ./RE-01 \
  entradas/valida.cfg entradas/documento.txt sesion-re01/T1-traza.txt \
  > sesion-re01/T1-traza.stdout 2> sesion-re01/T1-traza.stderr
printf '%s\n' "$?" > sesion-re01/T1-traza.exit
```

Relacione aperturas, descriptores, lecturas, escrituras y cierre con las rutas reales. Distinga los archivos del cargador de los datos de la utilidad. Repita para T4 con una salida nueva y contraste si se abre. Después intente reutilizar un resultado existente: registre retorno y compruebe que conserva su hash.

Para la intervención RE-01, use T4. Localice el punto posterior a validar la configuración y anterior a consumir su límite. Guarde el valor y cambie solo el límite 16 a 17 en memoria. No cambie la longitud del registro ni el fichero. Compare con controles nuevos antes y después. Documente por qué intervenir en la transformación sería demasiado tarde para T4. El docente dispone de una sesión verificada con símbolos externos; la entrega debe justificar la localización propia.

Entregue las cinco pruebas, cálculo de base, logs, control de no sobrescritura, intervención y especificación revisada. Un fallo de ptrace debe resolverse en la VM autorizada o registrarse como tarea no ejecutada, sin inventar una observación.